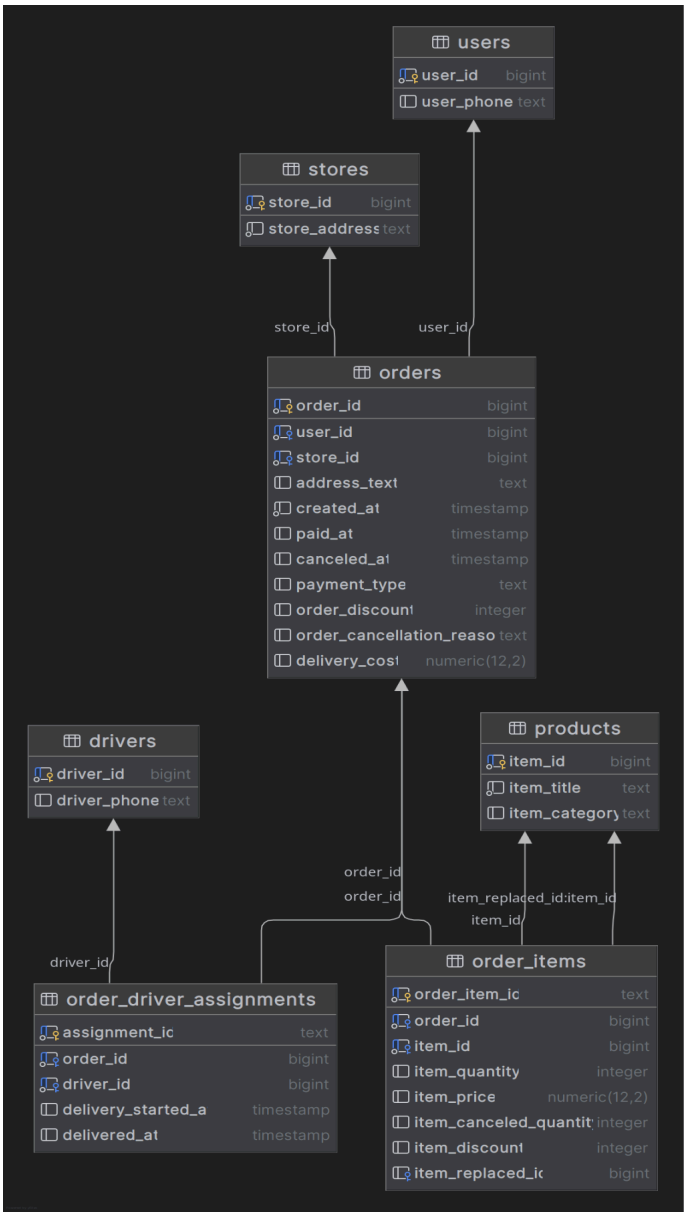
Итоговый проект по дисциплине “Python для инженерии данных”. Выполнили студенты 1 курса ОП “Инженерия данных”: Брагин Павел, Врублевский Александр, Маевский Дмитрий, Савостин Иван.

1. Схема данных

Используется сокращённая 3НФ: выделены основные сущности, которые убирают сильное дублирование, но схема не перегружена мелкими справочниками.

2. Как выглядит нормализованная схема данных



3. Таблицы

- **users** – пользователи, сделавшие заказ.
  - user_id: BIGINT, первичный ключ, идентификатор пользователя
  - user_phone: TEXT, телефон пользователя

- **drivers –** курьеры.
  - driver_id: BIGINT, первичный ключ, идентификатор курьера
  - driver_phone: TEXT, телефон курьера

- **stores –** магазины.
  - store_id: BIGINT, первичный ключ, идентификатор магазина
  - store_address: TEXT, адрес магазина

- **products** - товары
  - item_id: BIGINT, первичный ключ, идентификатор товара
  - item_title: TEXT, название товара
  - item_category: TEXT, категория товара

- **orders -** сущность верхнего уровня для заказов
  - order_id: BIGINT, первичный ключ, идентификатор заказа
  - user_id: BIGINT, внешний ключ к users.user_id, пользователь, сделавший заказ
  - store_id: BIGINT, внешний ключ к stores.store_id, магазин, из которого собран заказ
  - address_text: TEXT, адрес доставки
  - created_at: TIMESTAMP, время создания заказа
  - paid_at: TIMESTAMP, время оплаты заказа
  - canceled_at: TIMESTAMP, время отмены заказа
  - payment_type: TEXT, тип оплаты
  - order_discount: INT, скидка на заказ в процентах
  - order_cancellation_reason: TEXT, причина отмены заказа
  - delivery_cost: BIGINT, стоимость доставки

- **order_items** - позиции заказов

- order_item_id: TEXT, первичный ключ, суррогатный идентификатор позиции заказа
- order_id: BIGINT, внешний ключ к orders.order_id, идентификатор заказа
- item_id: BIGINT, внешний ключ к products.item_id, идентификатор товара
- item_quantity: BIGINT, количество заказанного товара
- item_price: BIGINT, цена за единицу товара
- item_canceled_quantity: BIGINT, отменённое количество товара
- item_discount: INT, скидка на товар в процентах
- item_replaced_id: BIGINT, внешний ключ к products.item_id, идентификатор товара-замены

- **order_driver_assignments** – назначенные на заказы курьеры
  - assignment_id: TEXT, первичный ключ, суррогатный идентификатор назначения
  - order_id: BIGINT, внешний ключ к orders.order_id, идентификатор заказа
  - driver_id: BIGINT, внешний ключ к drivers.driver_id, идентификатор курьера
  - delivery_started_at: TIMESTAMP, время начала доставки
  - delivered_at: TIMESTAMP, время завершения доставки

4. Связи между таблицами

- Один пользователь может иметь много заказов. Связь один-ко-многим между таблицами users и orders.

- Один магазин может обслуживать много заказов. Связь один-ко-многим между таблицами stores и orders.

- Один заказ может содержать много позиций. Связь один-ко-многим между таблицами orders и order_items.

- Один товар может встречаться во многих позициях заказов. Связь один-ко-многим между таблицами products и order_items.

- Один заказ может иметь несколько назначений курьеров. Связь один-ко-многим между таблицами orders и order_driver_assignments.

- Один курьер может быть назначен на много заказов. Связь один-ко-многим между таблицами drivers и order_driver_assignments.

5. Обоснование

Выбрана сокращённая 3НФ:

1) вынесены основные сущности: пользователь, курьер, магазин, товар, заказ, позиция заказа;
2) не выделялись отдельные справочники для `payment_type`, `item_category`, `order_cancellation_reason`, чтобы не усложнять схему;
3) история смены курьера хранится отдельно в `order_driver_assignments`.

6. Создание схемы данных (Скрипт)

```
CREATE SCHEMA IF NOT EXISTS dwh AUTHORIZATION warehouse_user;

CREATE TABLE IF NOT EXISTS dwh.users (
   user_id BIGINT PRIMARY KEY,
   user_phone TEXT
);

CREATE TABLE IF NOT EXISTS dwh.drivers (
   driver_id BIGINT PRIMARY KEY,
   driver_phone TEXT
);

CREATE TABLE IF NOT EXISTS dwh.stores (
   store_id BIGINT PRIMARY KEY,
   store_address TEXT NOT NULL
```

```
);

CREATE TABLE IF NOT EXISTS dwh.products (
    item_id BIGINT PRIMARY KEY,
    item_title TEXT NOT NULL,
    item_category TEXT
);

CREATE TABLE IF NOT EXISTS dwh.orders (
    order_id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    store_id BIGINT NOT NULL,
    address_text TEXT,
    created_at TIMESTAMP NOT NULL,
    paid_at TIMESTAMP,
    canceled_at TIMESTAMP,
    payment_type TEXT,
    order_discount INT,
    order_cancellation_reason TEXT,
    delivery_cost NUMERIC(12, 2),
    CONSTRAINT fk_orders_user
        FOREIGN KEY (user_id) REFERENCES dwh.users(user_id),
    CONSTRAINT fk_orders_store
        FOREIGN KEY (store_id) REFERENCES dwh.stores(store_id)
);

CREATE TABLE IF NOT EXISTS dwh.order_items (
    order_item_id TEXT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    item_id BIGINT NOT NULL,
    item_quantity INT,
    item_price NUMERIC(12, 2),
    item_canceled_quantity INT,
    item_discount INT,
    item_replaced_id BIGINT,
```

```
    CONSTRAINT fk_order_items_order
        FOREIGN KEY (order_id) REFERENCES dwh.orders(order_id),
    CONSTRAINT fk_order_items_product
                FOREIGN KEY (item_id) REFERENCES
dwh.products(item_id),
    CONSTRAINT fk_order_items_replaced_product
                FOREIGN KEY (item_replaced_id) REFERENCES
dwh.products(item_id)
);

CREATE TABLE IF NOT EXISTS dwh.order_driver_assignments (
    assignment_id TEXT PRIMARY KEY,
    order_id BIGINT NOT NULL,
    driver_id BIGINT NOT NULL,
    delivery_started_at TIMESTAMP,
    delivered_at TIMESTAMP,
    CONSTRAINT fk_assignments_order
        FOREIGN KEY (order_id) REFERENCES dwh.orders(order_id),
    CONSTRAINT fk_assignments_driver
        FOREIGN KEY (driver_id) REFERENCES dwh.drivers(driver_id)
);

CREATE INDEX IF NOT EXISTS idx_orders_user_id
    ON dwh.orders(user_id);

CREATE INDEX IF NOT EXISTS idx_orders_store_id
    ON dwh.orders(store_id);

CREATE INDEX IF NOT EXISTS idx_order_items_order_id
    ON dwh.order_items(order_id);

CREATE INDEX IF NOT EXISTS idx_order_items_item_id
    ON dwh.order_items(item_id);
```

```
CREATE INDEX IF NOT EXISTS idx_order_driver_assignments_order_id
  ON dwh.order_driver_assignments(order_id);

CREATE INDEX IF NOT EXISTS idx_order_driver_assignments_driver_id
  ON dwh.order_driver_assignments(driver_id);
```

7. Фрагмент скрипта для создания двух витрин

```
sql = """
      CREATE SCHEMA IF NOT EXISTS mart AUTHORIZATION warehouse_user;

  CREATE TABLE IF NOT EXISTS mart.mart_order_delivery (
    order_id BIGINT PRIMARY KEY,
    order_date DATE NOT NULL,
    created_at TIMESTAMP NOT NULL,
    user_id BIGINT NOT NULL,
    store_id BIGINT NOT NULL,
    driver_id BIGINT,
    payment_type TEXT,
    items_count BIGINT,
    gross_amount NUMERIC(14, 2),
    item_discount_sum NUMERIC(14, 2),
    order_discount NUMERIC(14, 2),
    delivery_cost NUMERIC(14, 2),
    final_amount NUMERIC(14, 2),
    is_paid INT,
    is_cancelled INT,
    cancellation_reason TEXT,
    delivery_started_at TIMESTAMP,
    delivered_at TIMESTAMP,
    wait_minutes NUMERIC(14, 2),
    delivery_minutes NUMERIC(14, 2)
  );
```

```
CREATE TABLE IF NOT EXISTS mart.mart_product_sales (
    order_date DATE NOT NULL,
    store_id BIGINT NOT NULL,
    item_id BIGINT NOT NULL,
    item_title TEXT,
    item_category TEXT,
    orders_count BIGINT,
    ordered_qty BIGINT,
    canceled_qty BIGINT,
    fulfilled_qty BIGINT,
    gross_revenue NUMERIC(14, 2),
    item_discount_sum NUMERIC(14, 2),
    replaced_count BIGINT
);

TRUNCATE TABLE mart.mart_order_delivery;
TRUNCATE TABLE mart.mart_product_sales;
"""

hook.run(sql)
```